

SIT315 Module 1 – Interrupt-Based Sense–Think–Act Embedded System

Name: Garv Kohli

Student ID: Your Student ID

Task: Module 1 Comprehensive Practical Task

Date: 29 July 2026

1. Introduction

This project demonstrates the design and implementation of a simple interrupt-based embedded system using an Arduino Uno in the Wokwi simulator. The system is developed using the Sense–Think–Act model, which is a common approach used in embedded and real-time systems. Three push buttons are used to simulate a door sensor, a window sensor, and an emergency sensor. Different interrupt techniques are implemented to detect these events, including an external interrupt, pin change interrupts, and a Timer1 interrupt that generates a periodic heartbeat every second. When an interrupt occurs, the Arduino processes the sensor input, makes a decision based on the current system state, and performs the required action by controlling an alarm LED. The system also displays status updates and interrupt events in the Serial Monitor, making it easy to observe the behaviour of the program. This project demonstrates how interrupts improve system responsiveness by allowing the microcontroller to react immediately to events without continuously polling the sensors. It also provides practical experience in designing a simple real-time embedded application using interrupts and event-driven programming.

2. System Components

| Component               | Purpose                     |
|-------------------------|-----------------------------|
| Arduino Uno             | Main controller             |
| Push Button (Door)      | External interrupt sensor   |
| Push Button (Window)    | Pin change interrupt sensor |
| Push Button (Emergency) | Emergency interrupt sensor  |
| Red LED                 | Alarm indicator             |
| 220Ω Resistor           | Protects the LED            |

3. Sense–Think–Act Model

Sense

The Arduino senses input from three push buttons. The door button uses an external interrupt on pin D2, while the window and emergency buttons use pin change interrupts on pins D8 and D9. Timer1 also generates an interrupt every second.

Think

The Arduino checks which sensor is active. If the emergency button is pressed, the alarm is activated immediately. If both the door and window sensors are active together, the alarm also turns on. Otherwise, the system remains in a safe or warning state.

Act

The Arduino controls the red LED based on the decision. When an emergency is detected, the LED turns on to indicate the alarm condition. System information is also displayed in the Serial Monitor.

Figure 1 – Timer Interrupt

Figure 2 – Door External Interrupt

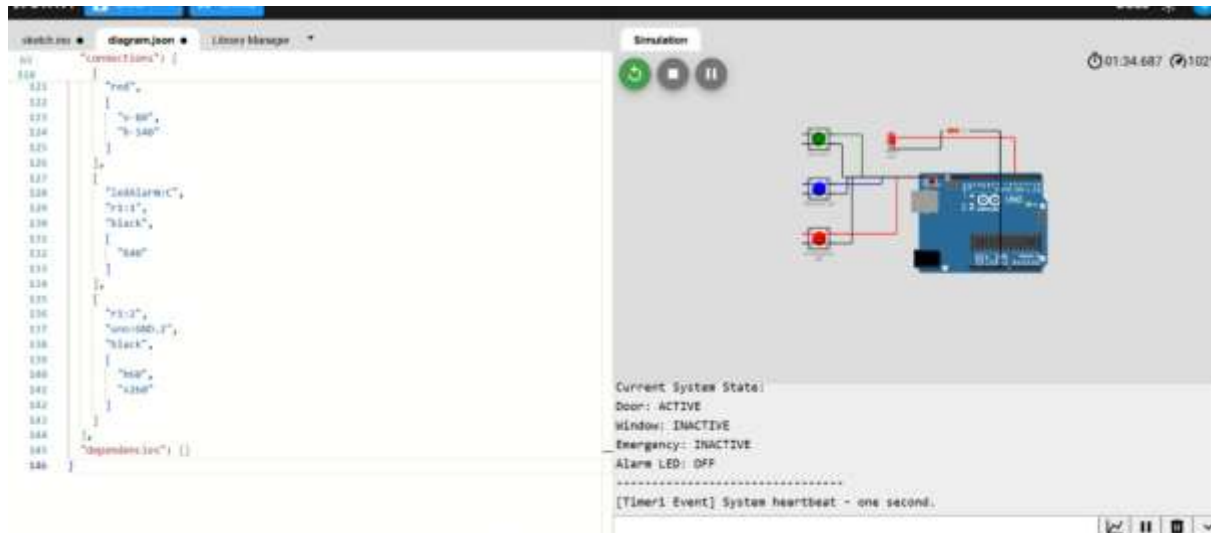


Figure 3 – Window Pin Change Interrupt

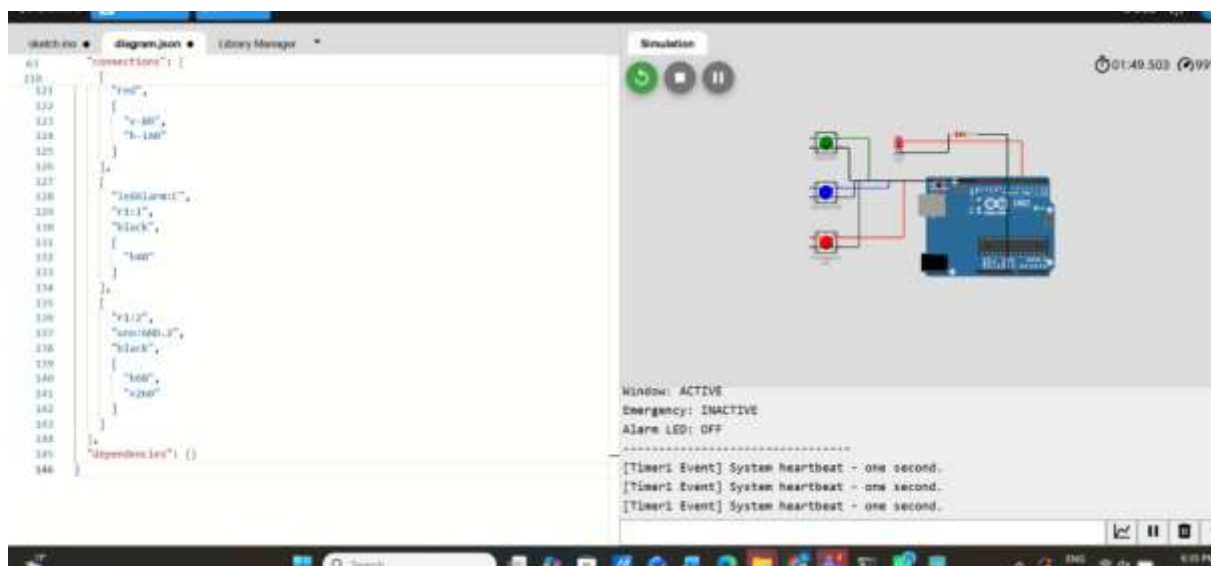
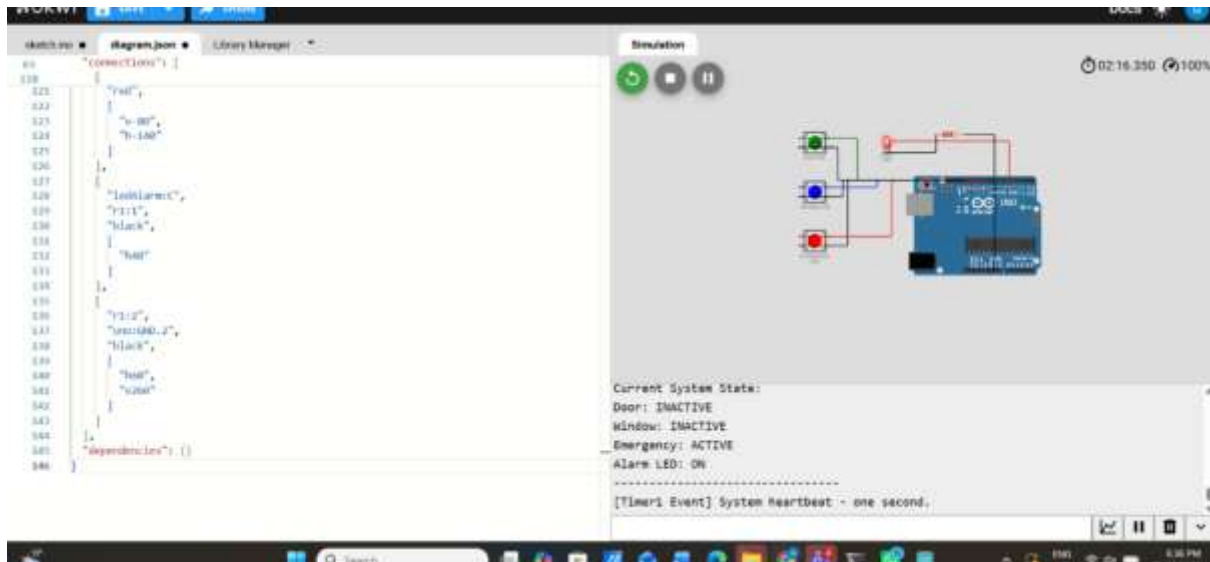


Figure 4 – Emergency Alarm



## 4. Interrupt Implementation

### External Interrupt (Door Sensor)

The door sensor is connected to digital pin D2 and uses an external interrupt with the `attachInterrupt()` function. When the button is pressed or released, the interrupt service routine (ISR) is triggered immediately. Instead of processing the event inside the ISR, the ISR only sets a flag. The main program checks this flag in the loop and processes the event safely. This approach keeps the interrupt routine short and improves the overall performance and reliability of the system.

### Pin Change Interrupt (Window and Emergency Sensors)

The window sensor on pin D8 and the emergency sensor on pin D9 use Pin Change Interrupts (PCINT). Unlike external interrupts, a single pin change interrupt group can monitor multiple pins at the same time. When either sensor changes state, the ISR records the event by setting a flag. The main program then reads the sensor status, updates the system state, and decides whether the alarm should remain OFF or turn ON. This method allows multiple sensors to be monitored efficiently using a single interrupt group.

### Timer Interrupt

Timer1 is configured to generate an interrupt every second using Compare Match mode (CTC). This periodic interrupt creates a heartbeat message in the Serial Monitor, showing that the system is running correctly even when no buttons are pressed. Timer interrupts are useful for performing regular tasks without blocking the main program or affecting the response to sensor events.

## 5. System Logic (Sense–Think–Act)

### Sense

The Arduino continuously waits for interrupt events instead of repeatedly checking each sensor. The door sensor uses an external interrupt, while the window and emergency sensors use pin change interrupts. Timer1 also generates a periodic interrupt every second to confirm that the system is operating correctly.

### Think

After an interrupt occurs, the program reads the sensor values and decides what action should be taken. If the emergency sensor is active, the alarm is activated immediately. If both the door and window sensors are active at the

same time, the alarm also turns ON. If only one normal sensor is active, the system remains in a warning state with the alarm turned OFF. If no sensors are active, the system stays in the safe state.

Act

Based on the decision, the Arduino controls the alarm LED. When an emergency or multiple sensor condition is detected, the LED turns ON to indicate an alarm. The system also prints the current sensor status, interrupt events, and system state in the Serial Monitor, making it easy to monitor how the embedded system responds to different situations.

6. Testing and Results

You can use this table in your report:

| Test                  | Expected Result                                         | Actual Result                                  | Status |
|-----------------------|---------------------------------------------------------|------------------------------------------------|--------|
| Door button (D2)      | External interrupt detected and Door becomes ACTIVE     | Door interrupt detected correctly              | Pass   |
| Window button (D8)    | Pin Change Interrupt detected and Window becomes ACTIVE | Window interrupt detected correctly            | Pass   |
| Emergency button (D9) | Alarm LED turns ON immediately                          | Emergency interrupt and alarm worked correctly | Pass   |
| Timer1                | Heartbeat message every second                          | Heartbeat displayed every second               | Pass   |
| Alarm Logic           | Alarm turns ON during emergency                         | Alarm LED turned ON correctly                  | Pass   |

7. Conclusion

This project successfully demonstrated an interrupt-based embedded system using the Arduino Uno and the Wokwi simulator. The system followed the Sense–Think–Act model to detect sensor inputs, process decisions, and control an alarm LED. Three sensors were implemented using different interrupt techniques, including an external interrupt, pin change interrupts, and a Timer1 interrupt. The timer interrupt generated a heartbeat message every second, while the sensor interrupts responded immediately when buttons were pressed. Testing showed that all interrupts worked correctly, and the alarm system behaved as expected. This project improved my understanding of interrupt-driven programming, event handling, and the importance of real-time responses in embedded systems.

8. Reflection

Through this project, I learned how interrupts make embedded systems more efficient by allowing the microcontroller to respond immediately to events instead of continuously checking sensor inputs. I gained practical experience in configuring external interrupts, pin change interrupts, and timer interrupts using Arduino. I also learned how the Sense–Think–Act model can be used to organise the logic of an embedded application in a simple and structured way. One challenge I faced was understanding how different interrupt types work and how to configure them correctly in the Wokwi simulator. After testing the system several times, I was able to identify and fix issues related to the button connections and interrupt behaviour. Overall, this project improved my confidence in developing interrupt-based applications and helped me understand how real-time embedded systems operate. The knowledge gained from this task will be useful in future IoT and embedded systems projects.

9. References (APA 7th Edition)

Arduino. (2024). attachInterrupt(). <https://www.arduino.cc/reference/en/language/functions/external-interrupts/attachinterrupt/>

Arduino. (2024). Interrupts. <https://docs.arduino.cc/learn/microcontrollers/interrupts>

Arduino. (2024). TimerOne Library Documentation. <https://www.arduino.cc/reference/en/>

Wokwi. (2024). Wokwi Arduino Simulator. <https://wokwi.com/>

Microchip Technology Inc. (2021). ATmega328P Datasheet.

<https://ww1.microchip.com/downloads/en/DeviceDoc/ATmega328P-DataSheet.pdf>